

how to monitor the changes in a property or its value and delay the initialization of stored property.

Computed properties work like methods. The difference lies in the re-run when one of the dependencies changes. In an example given below, the system will re-run when the array **ToDoItems** change.

To create a computed property, there would be a need to add a “**computed**” property to the component object, much like what is used in the “method” process.

```
• <h2>{{ToDoItems.filter(item => item.done).length}} out of
  {{ToDoItems.length}} items completed</h2>
```

Here, we have two items completed out of a total of five items on our to-do list. Our summary could read as “two items completed out of five”.

Add the code to your app **component** object in the **methods** property. The list of the summary methods will get the number of the **ToDoItems** and return to the string reporting to this:

```
• computed: {
•   listSummary() {
•     const numberFinishedItems = this.ToDoItems.
filter(item => item.done).length
•     return `${numberFinishedItems} out of ${this.
ToDoItems.length} items completed`
•   }
• }
```

Now you can add - **{{listSummary}}** directly to your template. This will be added inside an **<h2>** element, above the elements written above ****. Now add an **id** and an **aria-labelledby** to assign the **<h2>** contents for the label of the element in ****

Now add the described **<h2>** and then update **** in the interior of your app's template. The template will now read like this:

```
• <h2 id="list-summary">{{listSummary}}</h2>
• <ul aria-labelledby="list-summary" class="stack-large">
•   <li v-for="item in ToDoItems" :key="item.id">
•     <to-do-item :label="item.label" :done="item.done"
:id="item.id"></to-do-item>
•   </li>
• </ul>
```